D.3.10. std::atomic<integral-type> specializations

The `std::atomic<`*integral-type*`>` specializations of the `std::atomic` class template provide an atomic integral data type for each fundamental integer type, with a comprehensive set of operations.

The following description applies to these specializations of the `std::atomic<>` class template:

```
std::atomic<char>
std::atomic<signed char>
std::atomic<unsigned char>
std::atomic<short>
std::atomic<unsigned short>
std::atomic<int>
std::atomic<unsigned>
std::atomic<long>
std::atomic<unsigned long>
std::atomic<long long>
std::atomic<unsigned long long>
std::atomic<wchar_t>
std::atomic<char16_t>
std::atomic<char32_t>
```

Instances of these specializations are not `CopyConstructible` or `CopyAssignable`, because these operations can't be performed as a single atomic operation.

Class definition

```
template<>
struct atomic<integral-type>
{
    atomic() noexcept = default;
    constexpr atomic(integral-type) noexcept;
    bool operator=(integral-type) volatile noexcept;

    atomic(const atomic&) = delete;
    atomic& operator=(const atomic&) = delete;
    atomic& operator=(const atomic&) volatile = delete;

    bool is_lock_free() const volatile noexcept;
    bool is_lock_free() const noexcept;
```